March 2026

Planning Stage Action Plan Report

Equipment and Resource Booking Management System

Stanislav Starishko
Computing: Software Development: Graded Unit 2

Contents

Introduction 3

Project Brief and Analysis 3

- **Refined Project Brief** 3
- **Project Aims and Objectives** 4
- **Background Research** 4
- **Functional Requirements** 4
- **Non-Functional Requirements** 5
- **Business Rules** 5
- **Resources and Materials** 6
  - ***Development Tools and Technologies*** *6*
  - ***Project Management and Documentation*** *6*
  - ***Hardware*** *6*
  - ***Information Sources*** *7*

Project Plan 7

- **Project Stages Overview** 7
- **Main Tasks and Resources** 9
  - ***Main Tasks by Stage*** *9*
  - ***Project Team and Labour Resources*** *10*
  - ***Tools and Materials*** *10*

Solution Plan: Analysis and Design 11

- **Analysis Approach** 11
- **Top-Level Use Case Model** 11
- **Domain Model** 12
  - ***Equipment*** *13*
  - ***Booking*** *13*
  - ***User*** *14*
  - ***Category*** *14*
  - ***Enums*** *14*
  - ***Class Summary*** *15*
  - ***Relationships*** *15*
- **Database Design** 15
  - ***Table: category*** *16*
  - ***Table: equipment*** *16*
  - ***Table: app_user*** *16*
  - ***Table: booking*** *17*
  - ***Relationships*** *17*
  - ***Design Decisions*** *17*

Key Workflow: Booking Process 17
Activity Diagram 17
Sequence Diagram 19
Interface Design 19
Site Map 20
Wireframes 20
Dashboard Wireframe 21
Equipment List Wireframe 21
Booking Form Wireframe 22
Conclusion 22

Introduction

This document is the Planning Stage Action Plan Report for my Graded Unit 2 project. The project involves designing and developing an Equipment and Resource Booking Management System: a web-based application for a small organisation to manage shared equipment, track availability and handle bookings. This report covers three areas: an analysis of the project brief with initial requirements, a formal project plan with schedules and resources, and the solution plan including analysis and design evidence for the proposed application.

Project Brief and Analysis

This section presents my interpretation of the project scope, refined and clarified from the approved Project Proposal based on background research and practical experience. It covers the problem context, project aims, initial functional and non-functional requirements, background research, and the resources and information sources I plan to use throughout the project.

Refined Project Brief

The system is intended for a **small organisation** such as a training centre, charity or event support team. Organisations like these typically manage a pool of shared equipment: laptops, projectors, cameras, tablets, audio gear and other specialist items. These resources are booked for different purposes, assigned to different staff members or departments, and occasionally taken out of service for maintenance or repair.

At present, this kind of information is often handled through separate spreadsheets, paper logs or inconsistent records. This leads to practical problems: **double-bookings**, unclear responsibility for issued items, no reliable way to check what is available, and poor visibility of upcoming demand. The system I am proposing aims to replace this with a single, structured application.

The **intended users** are administrative staff who manage equipment on a day-to-day basis. The system is not intended for external customers or public-facing use.

The system will work with four core entities:

Entity	Key Attributes
Equipment	ID, name, category, location, condition, availability status
Booking	ID, equipment ID, booked by, date from, date to, purpose, status
User / Staff	ID, name, role, department, contact details
Category	ID, name, description

The main operations the system needs to support:

- Register new equipment and maintain its details
- Create, amend and cancel bookings
- Check equipment availability and detect booking conflicts
- Update equipment status (available, booked, under maintenance, unavailable)
- Search and filter equipment and booking records
- Provide a dashboard view of current and upcoming bookings

System boundaries. The scope of this project covers **staff-side administration** of equipment and bookings. The following are explicitly outside scope: customer self-service portals, payment processing, integration with external systems, and multi-site synchronisation.

Project Aims and Objectives

The **primary aim** of this project is to design and develop a web-based application that improves how a small organisation records, manages and books shared equipment and resources.

A **secondary aim** is to demonstrate and consolidate the technical skills developed throughout the HND Software Development course, particularly in object-oriented design, systems development and independent problem-solving.

To achieve these aims, I have defined the following objectives:

1. Analyse the requirements of a small equipment booking and administration scenario.
2. Design a structured, object-oriented solution with clearly defined entities and relationships.
3. Develop a web application that supports the core administrative tasks identified in the project brief.
4. Implement persistent data storage using a relational database.
5. Apply appropriate validation and error handling to key operations such as booking creation and status changes.
6. Test the completed solution against the original requirements and evaluate how effectively it meets the project aims.

Background Research

The idea for this project comes from practical experience rather than purely academic research. Through my volunteer work at **Glasgow Film Theatre** and **Shaw Trust**, combined with my previous professional background in larger corporate environments, I have seen first-hand how small organisations handle shared resources. In most cases, equipment and bookings are tracked through a mix of spreadsheets, paper logs and informal communication. This works at a basic level, but once the number of items, staff members or booking requests grows, problems start to appear: double-bookings, unclear ownership, items going missing, and no easy way to see what is available at a given time. This real-world observation is what shaped the brief for this project.

In terms of common patterns, most resource management systems I looked at share a similar structure: a catalogue of items with status tracking, a date-based booking mechanism with conflict detection, and a simple workflow for marking items as available, reserved or out of service. I plan to follow this pattern, focusing on the core booking and status management rather than advanced features like automated overdue alerts or calendar integrations, which fall outside the current scope.

For the **technology stack**, my initial consideration was a desktop application using JavaFX with SQLite for local storage. After further research, I moved away from this approach. A desktop solution would not match the real-world scenario described in the brief, where multiple staff members need to access the system from different workstations. Instead, I chose **Spring Boot** for the back-end, **Thymeleaf** for server-side HTML rendering, and a **relational SQL database** for persistent storage. This combination supports a browser-based application that is closer to what a real organisation would use, while also giving me the opportunity to work with frameworks and libraries I have not used before, which is one of the assessment requirements for the Development Stage.

For the database layer, I chose **H2**, a lightweight embedded SQL database that runs inside the application without requiring separate installation. This simplifies the development setup, while the use of JPA (Java Persistence API) means the system can be switched to any other SQL database without changes to the application code.

Functional Requirements

The following initial functional requirements have been identified based on the project brief. These will guide the design and implementation of the system.

ID	Area	Requirement
FR-01	Equipment	The system shall allow staff to register new equipment with details including name, category, location, condition and availability status.
FR-02	Equipment	The system shall allow staff to view, edit and delete existing equipment records.
FR-03	Equipment	The system shall allow staff to update the status of any equipment item (available, booked, under maintenance, unavailable).
FR-04	Booking	The system shall allow staff to create a new booking by selecting an equipment item, a date range and a purpose.
FR-05	Booking	The system shall check for booking conflicts before confirming a reservation and prevent double-bookings for the same item and period.
FR-06	Booking	The system shall allow staff to amend or cancel an existing booking.
FR-07	Booking	The system shall automatically update the equipment status to reflect active bookings.
FR-08	Search	The system shall allow staff to search equipment by name, category, location or availability status.
FR-09	Search	The system shall allow staff to search and filter bookings by date, equipment or staff member.
FR-10	Dashboard	The system shall provide a dashboard view showing current and upcoming bookings.
FR-11	Dashboard	The system shall display a summary of equipment availability across all categories.
FR-12	Category	The system shall allow staff to create, edit and delete equipment categories.

Non-Functional Requirements

ID	Area	Requirement
NFR-01	Usability	The interface shall be clear and intuitive, allowing staff to complete core tasks without specialist training.
NFR-02	Reliability	The system shall handle invalid input and unexpected conditions gracefully, without crashing or losing data.
NFR-03	Performance	Pages shall load and respond within 2 seconds under normal usage conditions.
NFR-04	Maintainability	The code shall follow a clear structure with consistent naming conventions, appropriate comments and separation of concerns (MVC pattern).
NFR-05	Security	The system shall implement basic authentication to restrict access to authorised staff only.
NFR-06	Portability	The system shall run in any modern web browser without requiring additional plugins or client-side installation.

Business Rules

The following business rules define the key constraints and conditions that the system must enforce. These rules are derived from the project brief and will be referenced throughout the design and implementation.

ID	Rule	Applies to
BR-01	An equipment item cannot be booked if its status is set to maintenance or unavailable	Equipment, Booking
BR-02	Bookings for the same equipment item must not overlap by date range	Booking
BR-03	Every booking must be linked to a registered staff member	Booking, User
BR-04	Every equipment item must belong to exactly one category	Equipment, Category
BR-05	A booking end date must be equal to or later than its start date	Booking
BR-06	A category can only be deleted if no equipment items are currently assigned to it	Category, Equipment
BR-07	An equipment item can only be deleted if it has no active bookings	Equipment, Booking
BR-08	Equipment status must automatically update to reflect active bookings	Equipment, Booking

Resources and Materials

The following resources and materials are required for the project. All tools and technologies selected are either free, open-source or available through college provision, so the project carries no direct software costs.

Development Tools and Technologies

Resource	Purpose	How Obtained	Cost
IntelliJ IDEA	Primary IDE for Java development	JetBrains free educational licence	£0
Spring Boot	Back-end web application framework	Open-source (spring.io)	£0
Thymeleaf	Server-side HTML template engine	Open-source (bundled with Spring Boot)	£0
H2 Database	Relational SQL database for development and testing	Open-source (bundled with Spring Boot)	£0
GitHub	Version control and source code repository	Free account	£0
Apache Maven	Build automation and dependency management	Open-source (bundled with IntelliJ)	£0

Project Management and Documentation

Resource	Purpose	How Obtained	Cost
MS Project	Project scheduling, Gantt chart and resource planning	College licence	£0
Microsoft Word	Document preparation and report writing	College licence	£0

Hardware

Resource	Purpose	How Obtained	Cost
Personal laptop	Development, testing and documentation	Already owned	£0
Internet access	Research, version control and framework downloads	Home broadband	£0

Information Sources

The following sources will be used throughout the project for research, learning and reference.

Source	Purpose
Spring Boot Official Documentation (spring.io)	Framework setup, configuration, data access, validation
Thymeleaf Documentation (thymeleaf.org)	Template syntax, form handling, layout features
Baeldung (baeldung.com)	Practical tutorials on Spring Boot, JPA, testing
H2 Database Documentation (h2database.com)	Database configuration, SQL syntax, console usage
OMG Unified Modelling Language Specification (uml.org)	Standard notation for use case, class and activity diagrams
Entity-Relationship Model references	Database design notation and conventions
SQA Unit Specification (H48W 35)	Assessment requirements, evidence criteria, grading guidance
HND Course Materials	Lecture notes on OOP, data structures, analysis and design
Stack Overflow (stackoverflow.com)	Troubleshooting specific technical issues during development
GitHub Documentation (docs.github.com)	Version control workflows and repository management

Project Plan

This section outlines the formal plan for the project, including the stage structure, schedule, milestones, deliverables and resources. The detailed project schedule was prepared in MS Project and is attached separately as supporting evidence (see **Project_Plan_Stanislav_Starishko.mpp**).

Project Stages Overview

The project follows three stages as defined by the Graded Unit 2 assessment structure.

Planning Stage covers the analysis of the project brief, identification of requirements, project scheduling and initial solution design. The output is this Action Plan Report.

Development Stage covers the technical implementation: setting up the environment, building the application, testing and preparing documentation. The output is a development portfolio with working code, test evidence and user documentation.

Evaluation Stage covers a review of the completed solution against the original brief, an assessment of strengths and weaknesses, recommendations for future improvements and a reflection on the skills gained. The output is an evaluation report.

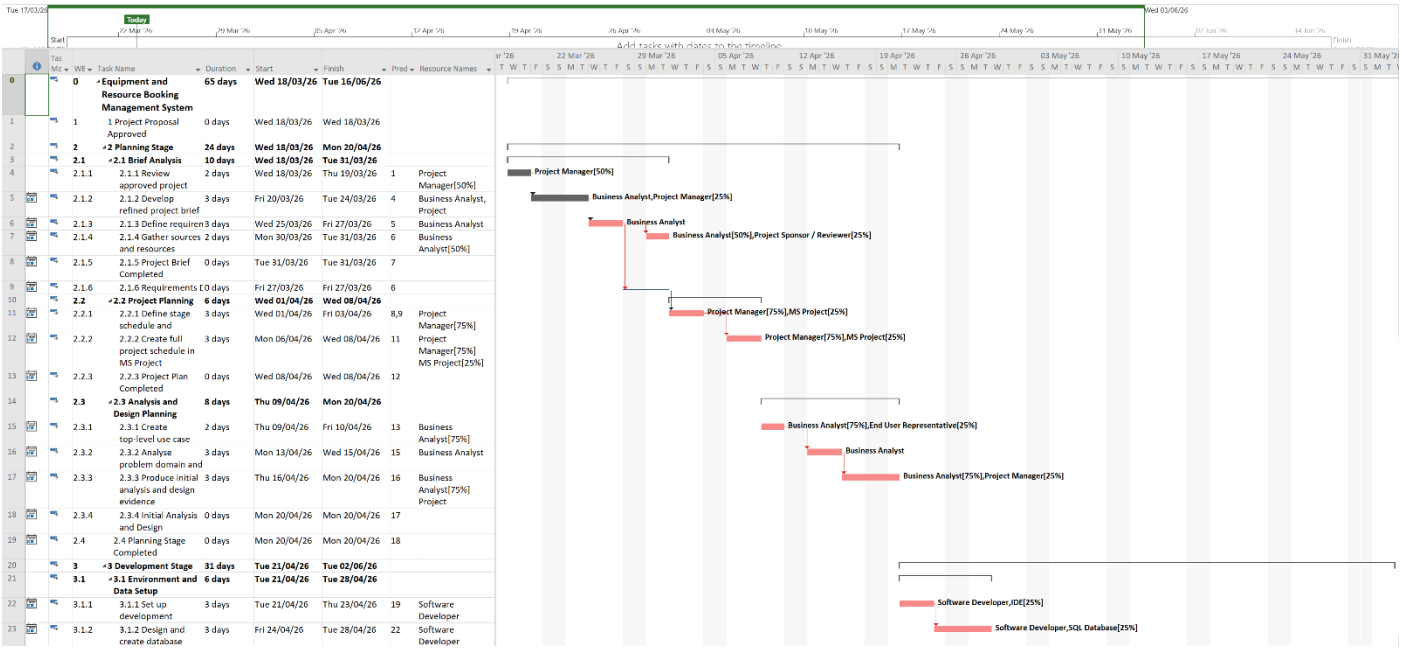
The overall project runs from 18 March to 17 June 2026, with each stage building directly on the deliverables of the previous one.

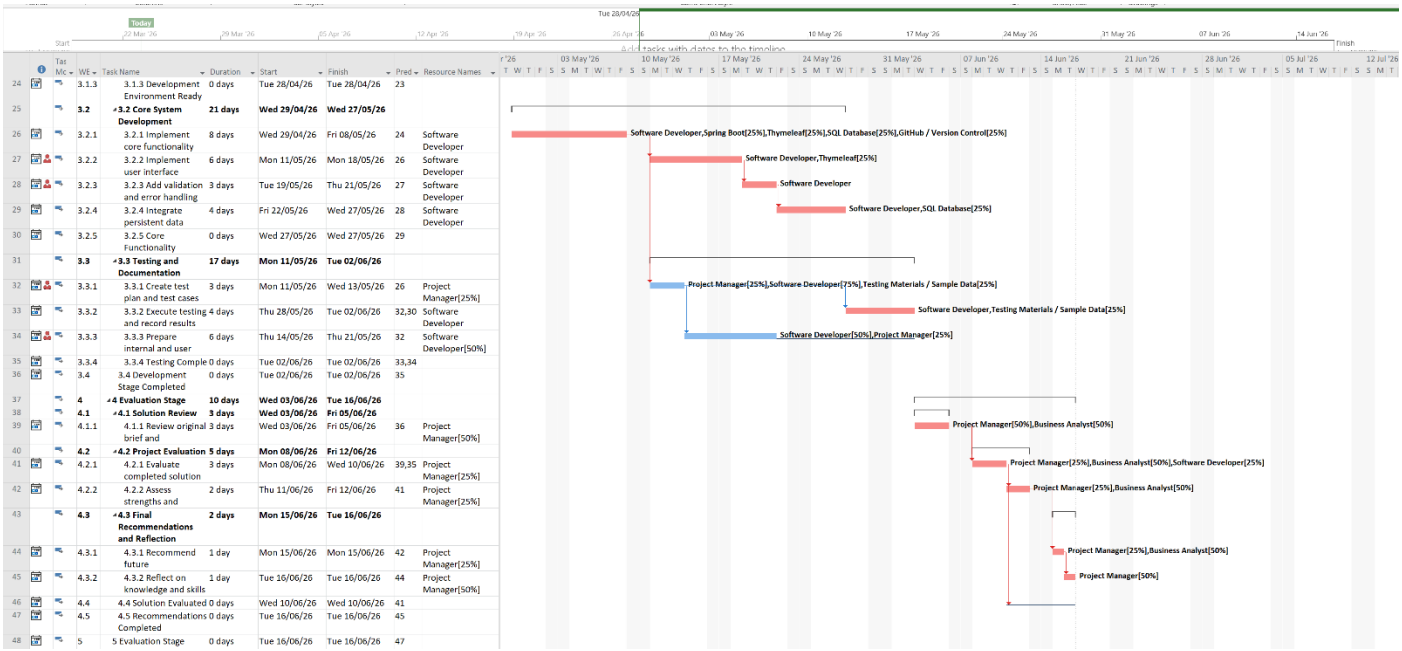
Schedule, Milestones and Deliverables

The table below summarises the key milestones and deliverables across all three stages of the project.

Stage	Milestone	Target Date	Deliverable
Planning	Requirements Defined	27 Mar 2026	Functional and non-functional requirements
Planning	Project Brief Completed	31 Mar 2026	Refined project brief
Planning	Project Plan Completed	08 Apr 2026	MS Project schedule
Planning	Initial Analysis and Design Completed	20 Apr 2026	Use case model, domain model, design evidence
Planning	**Planning Stage Completed**	**20 Apr 2026**	**Planning Stage Action Plan Report**
Development	Development Environment Ready	28 Apr 2026	Configured project with database
Development	Core Functionality Implemented	27 May 2026	Working application with core features
Development	Testing Completed	02 Jun 2026	Test plan, test results, user documentation
Development	**Development Stage Completed**	**02 Jun 2026**	**Development Stage Portfolio**
Evaluation	Solution Evaluated	10 Jun 2026	Evaluation findings
Evaluation	Recommendations Completed	16 Jun 2026	Future improvements and reflection
Evaluation	**Evaluation Stage Completed**	**16 Jun 2026**	**Evaluation Stage Report**

The figures below show the overall project Gantt chart from MS Project.





24		3.1.3	3.1.3 Development Environment Ready	0 days	Tue 28/04/26	Tue 28/04/26	23		
25		**3.2**	**3.2 Core System Development**	**21 days**	**Wed 29/04/26**	**Wed 27/05/26**			
26		3.2.1	3.2.1 Implement core functionality	8 days	Wed 29/04/26	Fri 08/05/26	24	Software Developer	
27		3.2.2	3.2.2 Implement user interface	6 days	Mon 11/05/26	Mon 18/05/26	26	Software Developer	
28		3.2.3	3.2.3 Add validation and error handling	3 days	Tue 19/05/26	Thu 21/05/26	27	Software Developer	
29		3.2.4	3.2.4 Integrate persistent data	4 days	Fri 22/05/26	Wed 27/05/26	28	Software Developer	
30		3.2.5	3.2.5 Core Functionality	0 days	Wed 27/05/26	Wed 27/05/26	29		
31		**3.3**	**3.3 Testing and Documentation**	**17 days**	**Mon 11/05/26**	**Tue 02/06/26**			
32		3.3.1	3.3.1 Create test plan and test cases	3 days	Mon 11/05/26	Wed 13/05/26	26	Project Manager[25%]	
33		3.3.2	3.3.2 Execute testing and record results	4 days	Thu 28/05/26	Tue 02/06/26	32,30	Software Developer	
34		3.3.3	3.3.3 Prepare internal and user	6 days	Thu 14/05/26	Thu 21/05/26	32	Software Developer[50%]	
35		3.3.4	3.3.4 Testing Comple	0 days	Tue 02/06/26	Tue 02/06/26	33,34		
36		3.4	3.4 Development Stage Completed	0 days	Tue 02/06/26	Tue 02/06/26	35		
37		**4**	**4 Evaluation Stage**	**10 days**	**Wed 03/06/26**	**Tue 16/06/26**			
38		**4.1**	**4.1 Solution Review**	**3 days**	**Wed 03/06/26**	**Fri 05/06/26**			
39		4.1.1	4.1.1 Review original brief and	3 days	Wed 03/06/26	Fri 05/06/26	36	Project Manager[50%]	
40		**4.2**	**4.2 Project Evaluation**	**5 days**	**Mon 08/06/26**	**Fri 12/06/26**			
41		4.2.1	4.2.1 Evaluate completed solution	3 days	Mon 08/06/26	Wed 10/06/26	39,35	Project Manager[25%]	
42		4.2.2	4.2.2 Assess strengths and	2 days	Thu 11/06/26	Fri 12/06/26	41	Project Manager[25%]	
43		**4.3**	**4.3 Final Recommendations and Reflection**	**2 days**	**Mon 15/06/26**	**Tue 16/06/26**			
44		4.3.1	4.3.1 Recommend future	1 day	Mon 15/06/26	Mon 15/06/26	42	Project Manager[25%]	
45		4.3.2	4.3.2 Reflect on knowledge and skills	1 day	Tue 16/06/26	Tue 16/06/26	44	Project Manager[50%]	
46		4.4	4.4 Solution Evaluated	0 days	Wed 10/06/26	Wed 10/06/26	41		
47		4.5	4.5 Recommendations Completed	0 days	Tue 16/06/26	Tue 16/06/26	45		
48		5	5 Evaluation Stage	0 days	Tue 16/06/26	Tue 16/06/26	47		

Main Tasks and Resources

The table below outlines the main tasks for each project stage. These form the basis of the detailed schedule in MS Project.

Main Tasks by Stage

Stage	Task Group	Key Tasks
Planning	Brief Analysis	Review approved proposal, develop refined brief, define requirements, gather sources
Planning	Project Planning	Define schedules and milestones, create full project schedule in MS Project
Planning	Analysis and Design	Create use case model, analyse problem domain, produce design evidence
Development	Environment Setup	Set up development environment, design and create database structure
Development	Core Development	Implement core functionality, implement user interface, add validation and error handling, integrate persistent storage
Development	Testing and Documentation	Create test plan and test cases, execute testing, prepare user documentation
Evaluation	Review and Evaluation	Review original brief, evaluate completed solution, assess strengths and weaknesses
Evaluation	Recommendations	Recommend future improvements, reflect on knowledge and skills developed

Project Team and Labour Resources

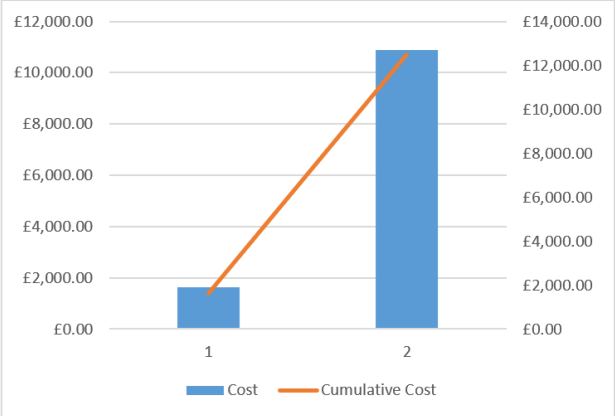
Since this is an academic project, the team structure is modelled as a small project simulation. Hourly rates were estimated based on median UK salary data from the National Careers Service, converted to hourly equivalents for planning purposes.

Role	Initials	Hourly Rate	Primary Responsibilities
Project Manager	PM	£24/hr	Planning, scheduling, documentation oversight, evaluation
Business Analyst	BA	£21/hr	Requirements analysis, use case modelling, domain analysis, evaluation
Software Developer	SD	£23/hr	Environment setup, coding, testing, internal documentation
Project Sponsor / Reviewer	SR	£28/hr	Review and approval at key milestones (limited involvement)
End User Representative	EU	£16/hr	Input on requirements and usability feedback (limited involvement)

Based on the planned resource allocation in MS Project, the estimated total project cost is approximately £12,500. A detailed cost breakdown by stage and resource is available in the attached project file.

CASH FLOW

Actual Cost	Baseline Cost	Remaining Cost	Cost Variance
£0.00	£0.00	£12,498.00	£12,498.00



The chart shows the project's cumulative cost and the cost per quater.
To see the costs for a different time period, select the Edit option from the Field List.

The table below shows cost information for all top-level tasks.
To see cost stats for all tasks, set the Outline Level in the Field List.

Name	Remaining Cost	Actual Cost	Cost	ACWP	BCWP	BCWS
Project Proposal Approved	£0.00	£0.00	£0.00	£0.00	£0.00	£0.00
Planning Stage	£3,830.00	£0.00	£3,830.00	£0.00	£0.00	£0.00
Development Stage	£7,102.00	£0.00	£7,102.00	£0.00	£0.00	£0.00
Evaluation Stage	£1,566.00	£0.00	£1,566.00	£0.00	£0.00	£0.00
Evaluation Stage Completed	£0.00	£0.00	£0.00	£0.00	£0.00	£0.00
Project Completed	£0.00	£0.00	£0.00	£0.00	£0.00	£0.00

Tools and Materials

All tools and technologies used in the project are free, open-source or available through college provision. A full breakdown is provided in section «**Resources and Materials**». No direct project costs are associated with software or materials.

Solution Plan: Analysis and Design

This section covers how I plan to build the system, from analysis through to interface design. I have used an object-oriented approach throughout, with UML diagrams to model different aspects of the solution. Each subsection focuses on a specific layer: what the system does, how it is structured, how the data is stored, how the main booking process works, and what the interface will look like.

Analysis Approach

For this project I have chosen an **object-oriented analysis and design** approach, which fits naturally with Java and the Spring Boot framework. To model the solution I am using **UML**, which gives me a consistent way to describe different aspects of the system in a format that is widely understood.

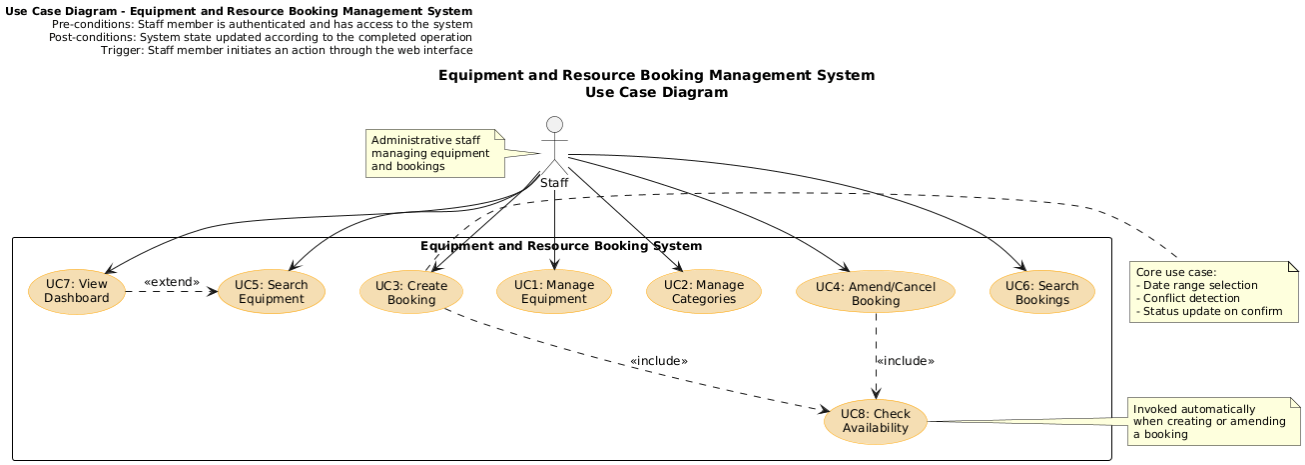
The specific techniques I have selected and what each one covers:

- **Use case diagram** to capture what the system does from the user's perspective, mapping directly to the functional requirements.
- **Class diagram** to define the core entities, their attributes and how they relate to each other.
- **ER diagram** to translate the domain model into a database structure with tables, keys and relationships.
- **Activity diagram** to walk through the booking process step by step, including the conflict detection logic.
- **Sequence diagram** to show how the system components (controller, service, repository) interact during the booking operation.
- **Site map and wireframes** to show how the interface is organised and what the key pages will look like.

Together, these cover the static structure (what is in the system), the dynamic behaviour (how it works) and the presentation layer (how it looks to the user). I kept the selection focused rather than producing every possible diagram, choosing only those that directly support the implementation.

Top-Level Use Case Model

The diagram below shows the top-level use case model for the system. There is a single actor, **Staff**, representing the administrative users who will interact with the system through a web browser. Eight use cases have been identified, mapping directly to the functional requirements defined in section «**Functional Requirements**».



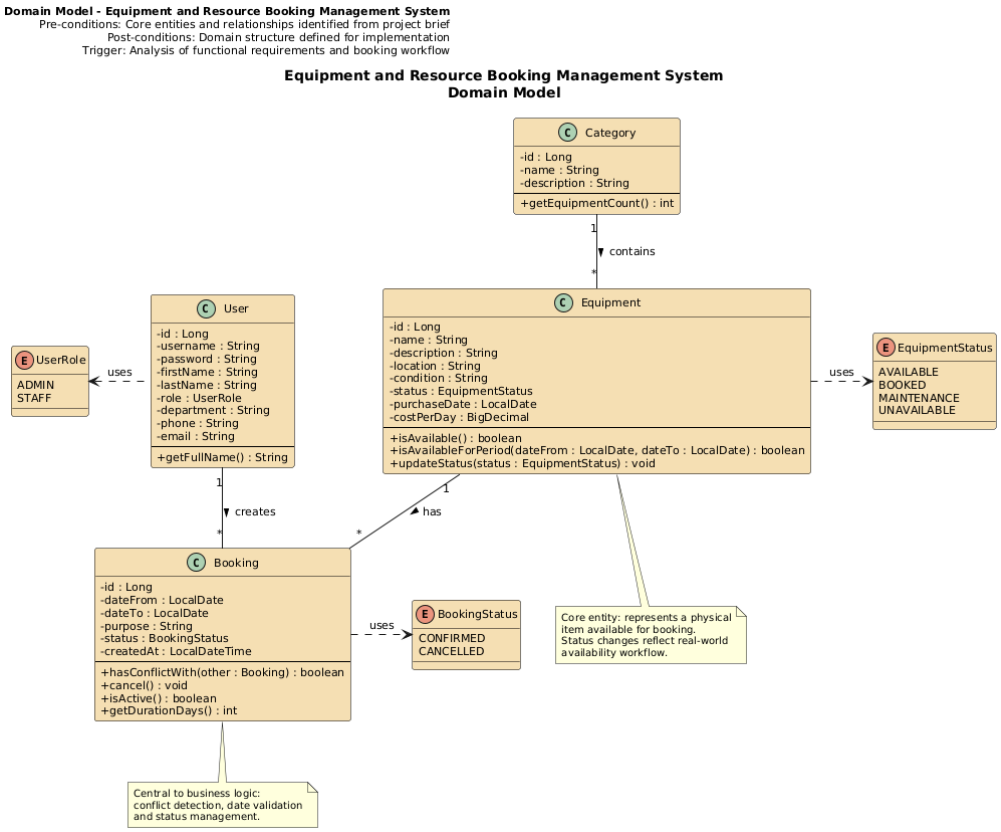
ID	Use Case	Description	Maps to
UC7	View Dashboard	View summary of current and upcoming bookings and equipment availability	FR-10, FR-11
UC5	Search Equipment	Search and filter equipment by name, category, location or status	FR-08

ID	Use Case	Description	Maps to
UC3	Create Booking	Create a new booking with date range, equipment and purpose selection	FR-04, FR-05
UC1	Manage Equipment	Add, edit, delete and update status of equipment items	FR-01, FR-02, FR-03
UC2	Manage Categories	Create, edit and delete equipment categories	FR-12
UC4	Amend/Cancel Booking	Modify or cancel an existing booking	FR-06
UC6	Search Bookings	Search and filter bookings by date, equipment or staff member	FR-09
UC8	Check Availability	Verify equipment availability and detect booking conflicts (included by UC3 and UC4)	FR-05

Together, these eight use cases cover all twelve functional requirements identified in the Functional Requirements section, ensuring full traceability between requirements and system behaviour.

Domain Model

The domain model below represents the core entities of the system, their attributes, methods and relationships. This is an analytical model derived from the project brief and functional requirements, not a final implementation class diagram. It focuses on what exists in the system and how the parts relate to each other.



The model is built around four main entities. **Equipment** is the central item being managed and booked. Each piece of equipment belongs to a **Category**, which allows grouping and filtering. **Booking** captures a reservation of a specific equipment item for a date range, and is always linked to the **User** who created it. Three enums define the constrained status values used across the system.

The key design decision here is that availability checking sits on the Equipment entity itself through isAvailableForPeriod(), while conflict detection between overlapping reservations is handled by hasConflictWith() on Booking. This keeps the logic close to the data it operates on, rather than pushing everything into a separate service layer at the domain level.

Equipment

Attribute	Type	Description
id	Long	Unique identifier
name	String	Equipment name
description	String	Brief description of the item
location	String	Physical location
condition	String	Current physical condition
status	EquipmentStatus	Availability status
purchaseDate	LocalDate	Date of purchase
costPerDay	BigDecimal	Daily booking cost

Method	Return Type	Description
isAvailable()	boolean	Checks if current status is AVAILABLE
isAvailableForPeriod(dateFrom, dateTo)	boolean	Checks for booking conflicts in a date range
updateStatus(status)	void	Changes the equipment status

Booking

Attribute	Type	Description
id	Long	Unique identifier
dateFrom	LocalDate	Booking start date
dateTo	LocalDate	Booking end date
purpose	String	Reason for booking
status	BookingStatus	Current booking status
createdAt	LocalDateTime	Timestamp of booking creation

Method	Return Type	Description
hasConflictWith(other)	boolean	Checks if date range overlaps with another booking
cancel()	void	Sets status to CANCELLED
isActive()	boolean	Returns true if status is CONFIRMED and dates are current

Method	Return Type	Description
getDurationDays()	int	Calculates number of days in the booking period

User

Attribute	Type	Description
id	Long	Unique identifier
username	String	Login username
password	String	Encrypted password
firstName	String	First name
lastName	String	Last name
role	UserRole	Access level
department	String	Department name
phone	String	Contact phone number
email	String	Contact email

Method	Return Type	Description
getFullName()	String	Returns firstName + lastName

Category

Attribute	Type	Description
id	Long	Unique identifier
name	String	Category name
description	String	Category description

Method	Return Type	Description
getEquipmentCount()	int	Returns number of equipment items in this category

Enums

Enum	Values	Used by
EquipmentStatus	AVAILABLE, BOOKED, MAINTENANCE, UNAVAILABLE	Equipment.status
BookingStatus	CONFIRMED, CANCELLED	Booking.status
UserRole	ADMIN, STAFF	User.role

Class Summary

Class	Purpose
Equipment	Represents a physical item available for booking
Booking	Represents a reservation of equipment for a specific date range
User	Represents a staff member who can create and manage bookings
Category	Groups equipment items by type for filtering and organisation

Relationships

Relationship	Type	Multiplicity	Description	Rule
Equipment → Category	Many-to-One	Many : One	Each equipment item belongs to one category	BR-04
Booking → Equipment	Many-to-One	Many : One	Each booking is for one equipment item	BR-01, BR-02
Booking → User	Many-to-One	Many : One	Each booking is created by one staff member	BR-03

Database Design

The ER diagram below shows how the domain model translates into a relational database structure. Each entity from section «**Domain Mode**» maps to a table, with attributes becoming columns and relationships implemented through foreign keys. Fields marked with an asterisk are mandatory (NOT NULL).

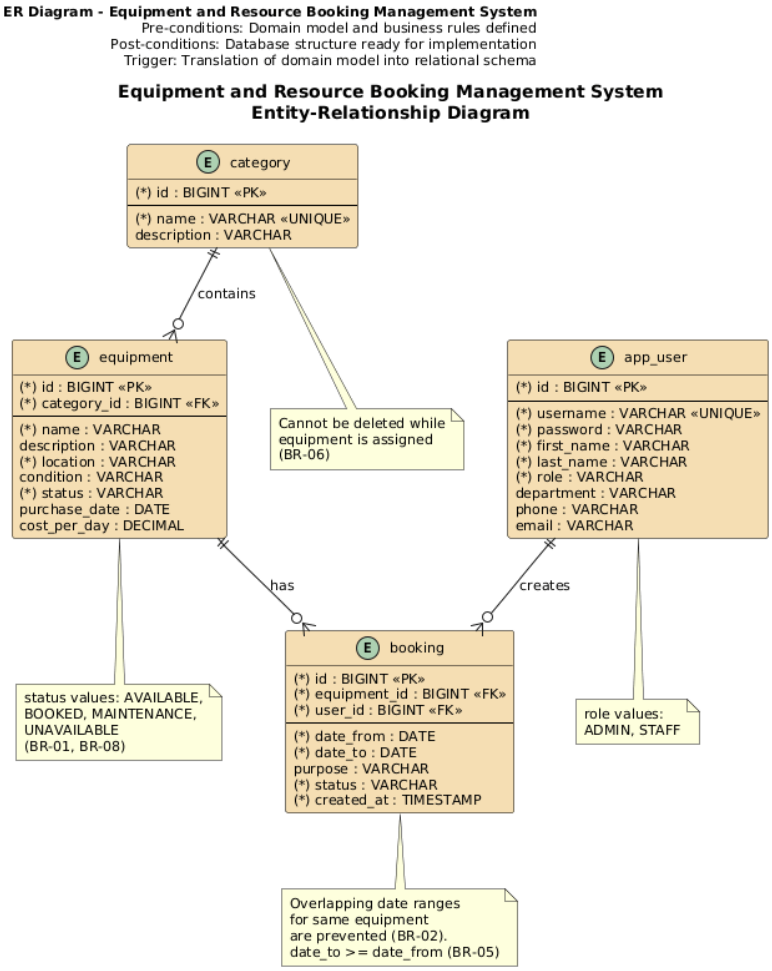


Table: category

Column	Type	Constraints	Description
id	BIGINT	PK, auto-generated	Unique identifier
name	VARCHAR	NOT NULL, UNIQUE	Category name
description	VARCHAR		Brief description

Table: equipment

Column	Type	Constraints	Description
id	BIGINT	PK, auto-generated	Unique identifier
name	VARCHAR	NOT NULL	Equipment name
description	VARCHAR		Brief description of the item
location	VARCHAR	NOT NULL	Physical location
condition	VARCHAR		Current physical condition
status	VARCHAR	NOT NULL	Availability status (BR-01, BR-08)
purchase_date	DATE		Date of purchase
cost_per_day	DECIMAL		Daily booking cost
category_id	BIGINT	FK → category.id, NOT NULL	Parent category (BR-04)

Table: app_user

Column	Type	Constraints	Description
id	BIGINT	PK, auto-generated	Unique identifier
username	VARCHAR	NOT NULL, UNIQUE	Login username
password	VARCHAR	NOT NULL	Encrypted password
first_name	VARCHAR	NOT NULL	First name
last_name	VARCHAR	NOT NULL	Last name
role	VARCHAR	NOT NULL	Access level (ADMIN, STAFF)
department	VARCHAR		Department name
phone	VARCHAR		Contact phone
email	VARCHAR		Contact email

Table: booking

Column	Type	Constraints	Description
id	BIGINT	PK, auto-generated	Unique identifier
date_from	DATE	NOT NULL	Booking start date (BR-05)
date_to	DATE	NOT NULL	Booking end date (BR-05)
purpose	VARCHAR		Reason for booking
status	VARCHAR	NOT NULL	Booking status (CONFIRMED, CANCELLED)
created_at	TIMESTAMP	NOT NULL	When the booking was created
equipment_id	BIGINT	FK → equipment.id, NOT NULL	Booked item (BR-02)
user_id	BIGINT	FK → app_user.id, NOT NULL	Staff member who created it (BR-03)

Relationships

Relationship	Type	Constraint	Business Rule
category → equipment	One-to-Many	equipment.category_id FK	BR-04, BR-06
equipment → booking	One-to-Many	booking.equipment_id FK	BR-01, BR-02, BR-07
app_user → booking	One-to-Many	booking.user_id FK	BR-03

Design Decisions

The table is named **app_user** rather than "user" because "user" is a reserved keyword in most SQL databases including H2 and PostgreSQL. Enum values such as equipment status and booking status are stored as **VARCHAR** columns rather than separate lookup tables; JPA maps Java enums directly to strings, which keeps the schema simple without losing type safety at the application level. Monetary values use **DECIMAL** rather than floating-point types to avoid rounding errors. Booking dates use **DATE** (day precision) while created_at uses **TIMESTAMP** (full precision), reflecting their different purposes. Cascade behaviour will be managed at the application layer: deleting a category is blocked if equipment is assigned (BR-06), and deleting equipment is blocked if active bookings exist (BR-07).

Key Workflow: Booking Process

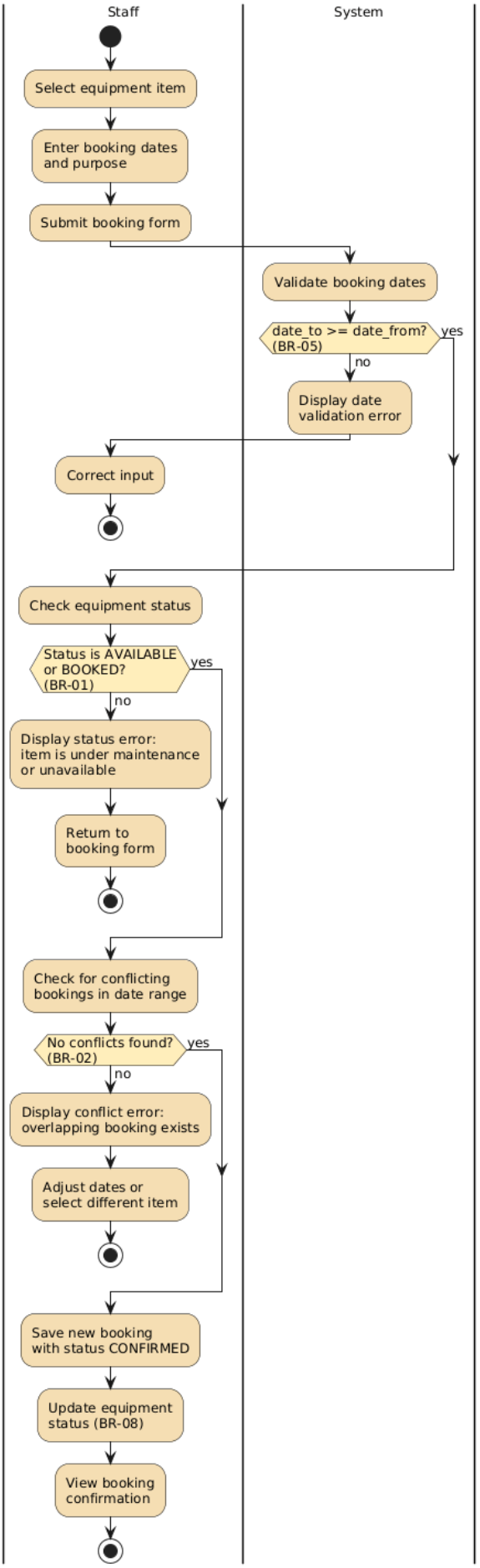
The booking process is the most complex workflow in the system. It involves multiple validation steps, conflict detection and automatic status updates. This section illustrates the process from two perspectives: an activity diagram showing the overall flow of decisions and actions, and a sequence diagram showing how the system components interact during the operation.

Activity Diagram

The activity diagram below shows the complete booking creation process from the staff member's initial action through to confirmation or rejection. Three key decision points enforce the business rules: date validation (BR-05), equipment status check (BR-01) and booking conflict detection (BR-02). If all checks pass, the booking is saved and the equipment status is updated automatically (BR-08).

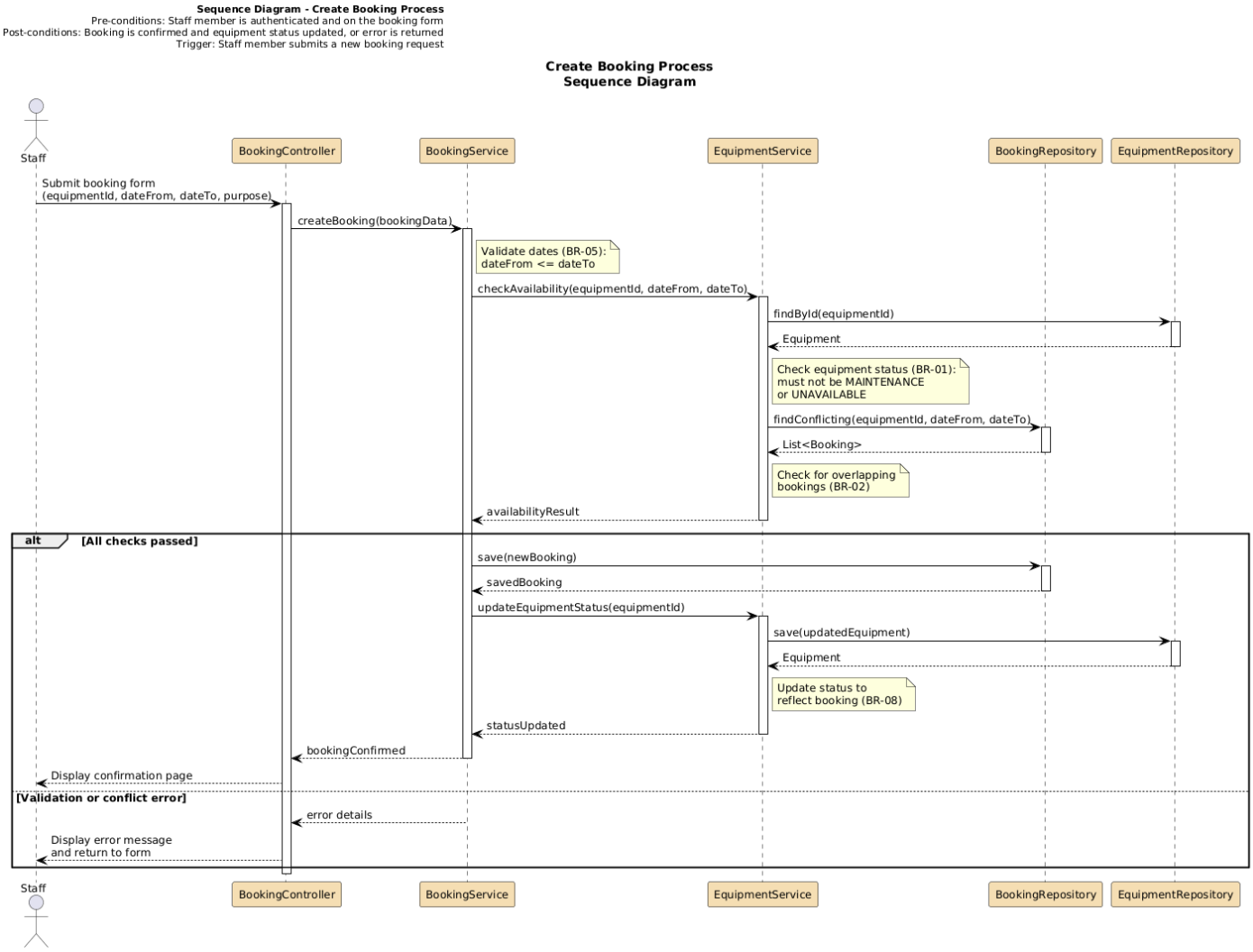
Activity Diagram - Create Booking Process
Pre-conditions: Staff member is authenticated and on the booking form
Post-conditions: Booking is confirmed and equipment status updated, or error is displayed
Trigger: Staff member submits a new booking request

Create Booking Process
Activity Diagram



Sequence Diagram

The sequence diagram below shows how the same booking process is handled across the system's components. The request flows from the controller through the service layer, where business logic is applied, down to the repository layer for data access. This separation follows the MVC pattern defined in NFR-04 and reflects the Spring Boot architecture chosen for the project.



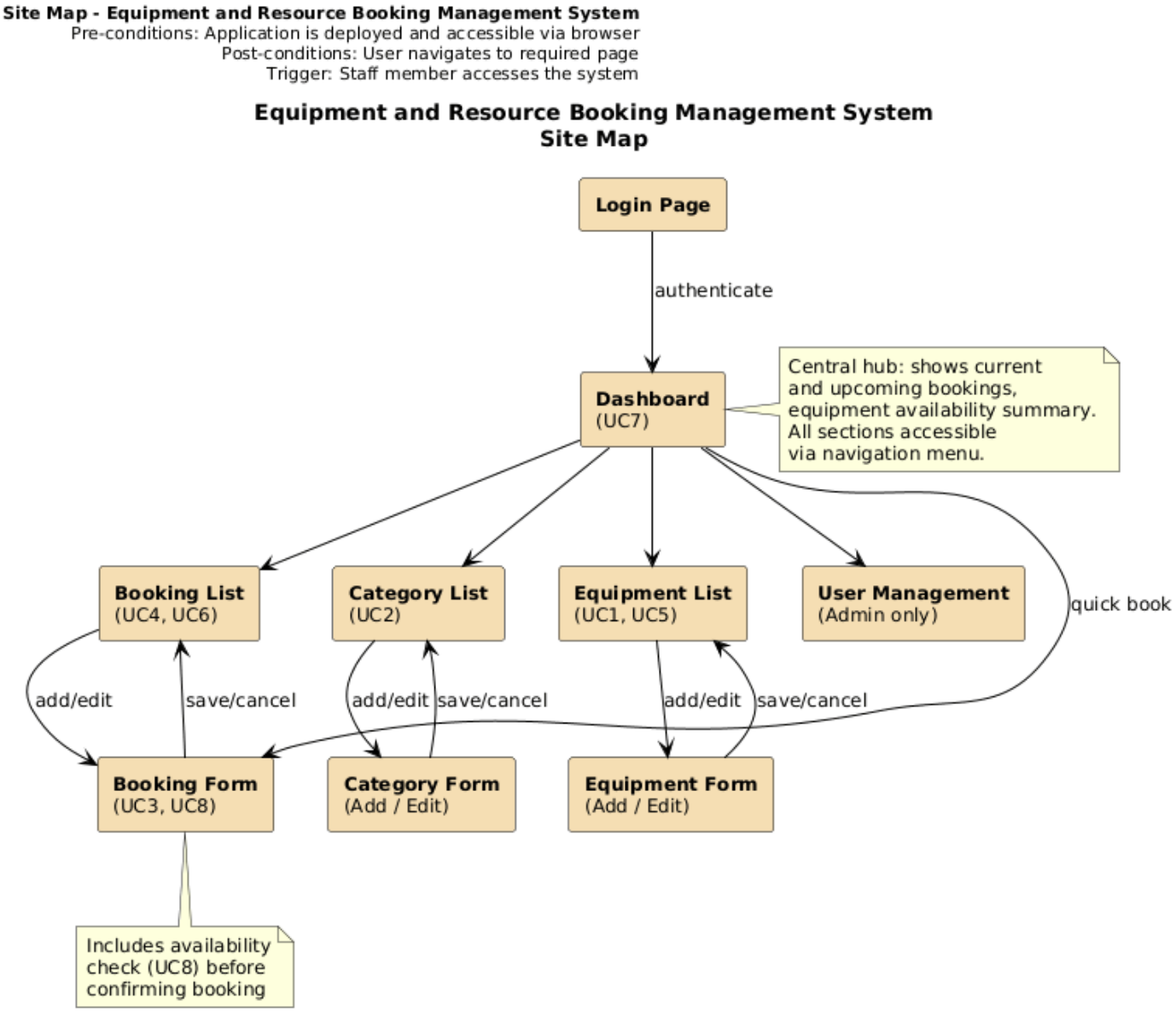
The diagram shows a clear separation of responsibilities: the controller handles the HTTP request, the booking service applies business logic (BR-05 date validation), the equipment service checks status (BR-01) and conflicts (BR-02), and the repositories manage data access. If all checks pass, the booking is saved and the equipment status is updated (BR-08). If any check fails, an error is returned to the controller, which displays the appropriate message to the staff member.

Interface Design

This section covers the overall structure and navigation of the application's user interface, followed by wireframe mockups of the key pages. The system is a browser-based application, so all interaction happens through standard web pages rendered by Thymeleaf templates.

Site Map

The site map below shows the main pages of the application and how users navigate between them. After logging in, the staff member lands on the Dashboard, which acts as the central hub. From there, all major sections are accessible through the main navigation menu. Each list page leads to its corresponding form for creating or editing records.

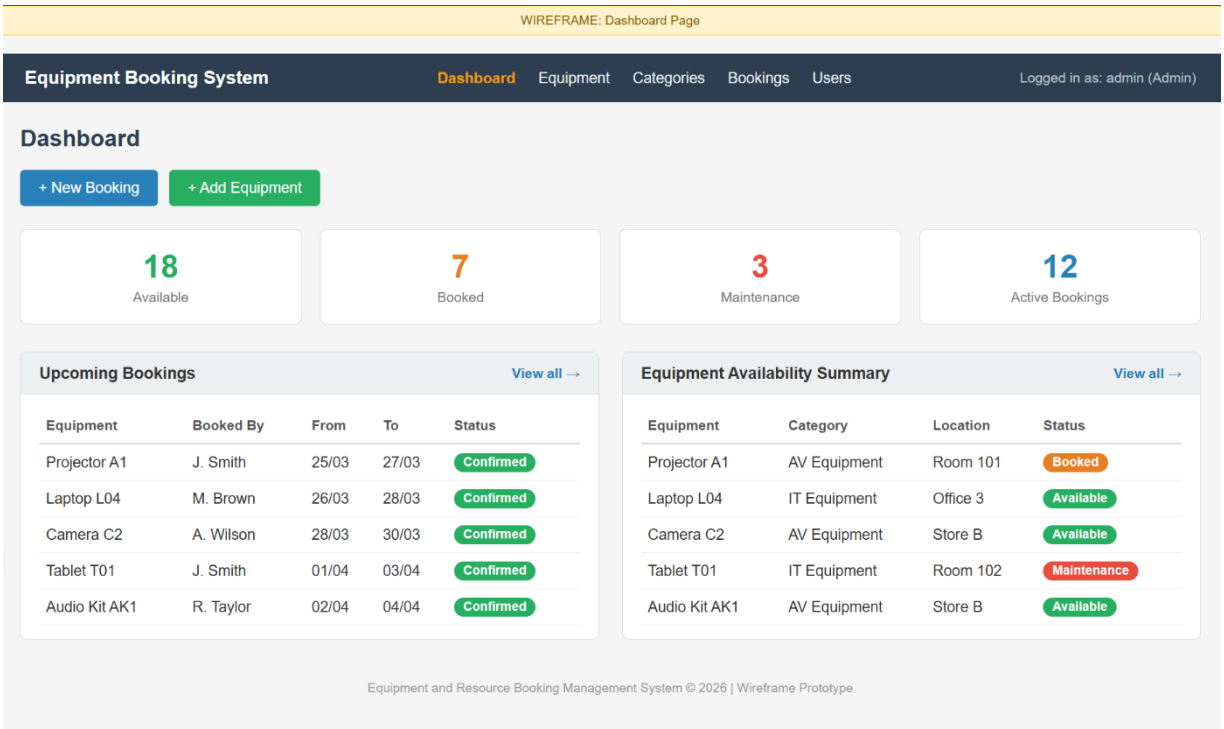


Wireframes

The wireframes below illustrate the layout and key elements of three representative pages: the Dashboard (overview page), the Equipment List (typical list page with search and filtering) and the Booking Form (the most complex form, involving equipment selection, date input and availability checking). These are not final designs but initial prototypes showing the intended structure and functionality.

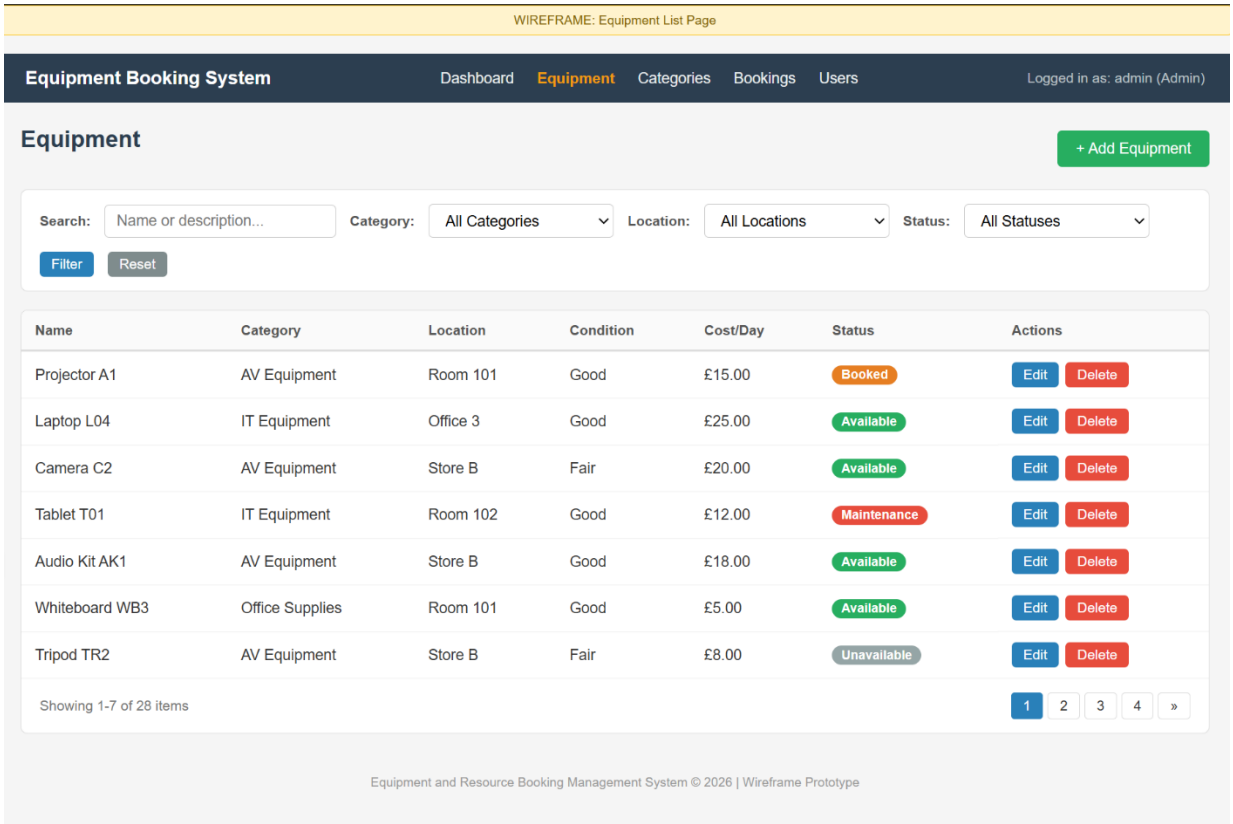
Dashboard Wireframe

The Dashboard provides an overview of equipment availability and upcoming bookings, with quick action buttons for common tasks (FR-10, FR-11).



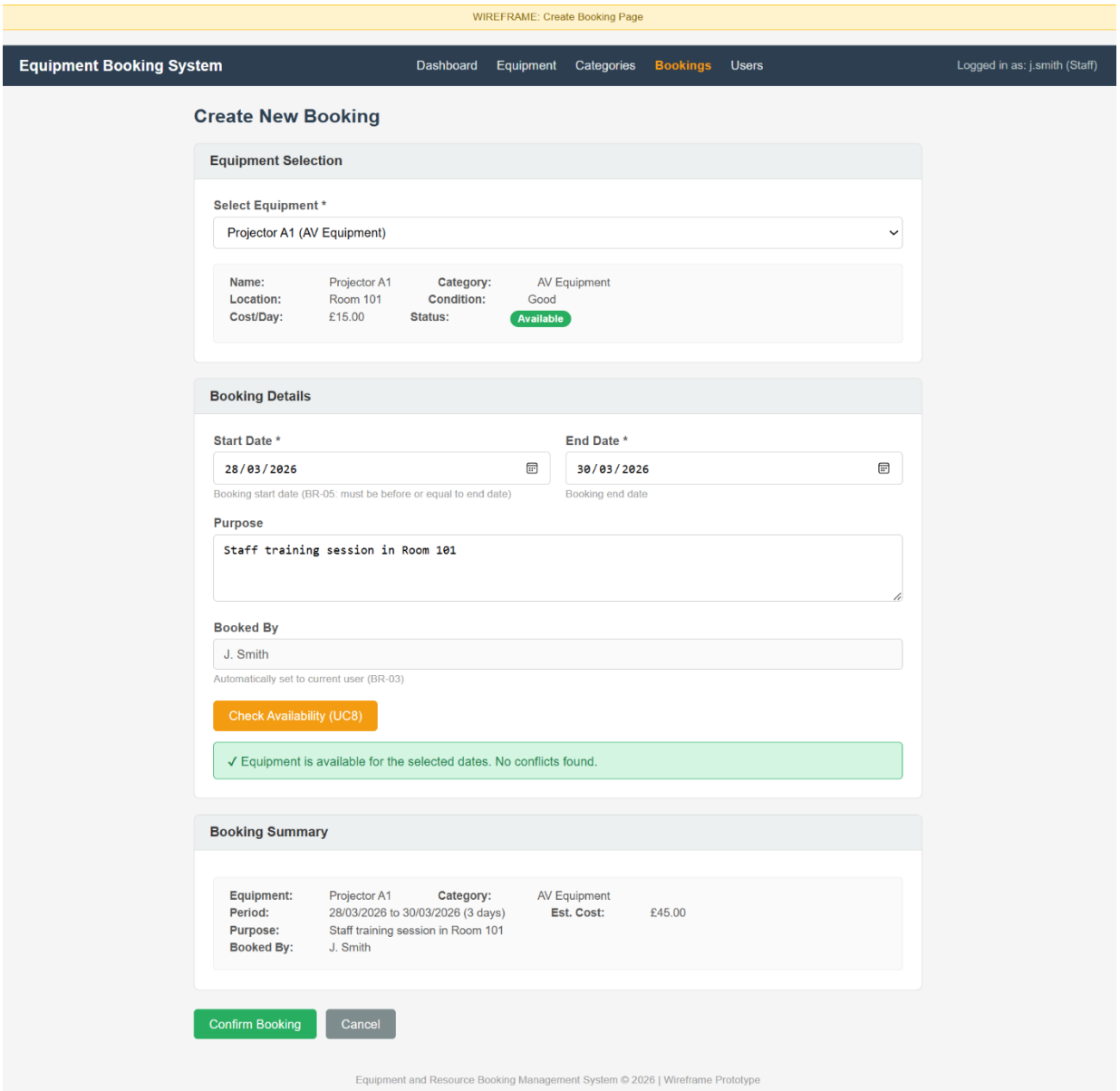
Equipment List Wireframe

The Equipment List page supports searching and filtering by name, category, location and status, with direct access to add, edit or delete records (FR-01, FR-02, FR-03, FR-08).



Booking Form Wireframe

The Booking Form is the most complex page, combining equipment selection, date input, availability checking (UC8) and a booking summary before confirmation (FR-04, FR-05).



Conclusion

This report has covered the full scope of the Planning Stage: from the refined project brief and requirements through to the project schedule, analysis and design evidence. The proposed solution is realistic and achievable within the project timescales, using a well-documented technology stack that I have not previously worked with, which will support both the learning objectives and the technical depth expected at this level. The design models, business rules and wireframes presented here provide a solid foundation for the Development Stage, where the focus will shift to implementation, testing and documentation.